



C PROGRAMMING REVIEW

COMPILATION

- Compilation is the translation of source code (the code we write) into object code (sequence of statements in machine language) by a **compiler**.
- The compilation process has four different steps:
 - The preprocessing
 - The compiling
 - The assembling
 - The linking

COMPILATION

- The compiler we will be using as an example is **gcc** which stands for **GNU Compiler Collection**.
- In order to use it, make sure you have it installed.
- Type `gcc -version` in your terminal. If you get the version displayed, then you are good to go. If not , install gcc first.
 - `sudo apt update && sudo apt upgrade` (*optional* to make sure you have an up to date system & ensure compatibility between packages)
 - `sudo apt install build-essential`

COMPILATION

- Let's assume we have a c file named “**test.c**” .
- In order for our test.c code to be executable, we need to enter the command “`gcc test.c`”, and the compiling process will go through all of the four steps it contains.
- Of course gcc has options that allow us to stop the compiling process after each step. Let's take a look at them.

STEP I - PREPROCESSING

The preprocessor has several roles:

- it gets rid of all the *comments* in the source file(s)
- it includes the code of the *header file(s)*, which is a file with extension `.h` which contains C function declarations
- In order to stop the compilation right after this step, we can use the option “-E” with the gcc command on the source file, and press enter.
- `gcc -E test.c`

STEP 2 - COMPILER

- The compiler will take the preprocessed file and generate Assembly code, so this will produce a “.s” file.
- We can stop after this step with the “-S” option on the gcc command, and press enter.
- `gcc -S test.c`

STEP 3 - ASSEMBLER

- The assembler takes transforms the code into object code, that is code in machine language (i.e. binary). This will produce a file ending in “.o”.
- We can stop the compilation process after this step by using the option “-c” with the gcc command, and pressing enter.
- `gcc -c test.c`

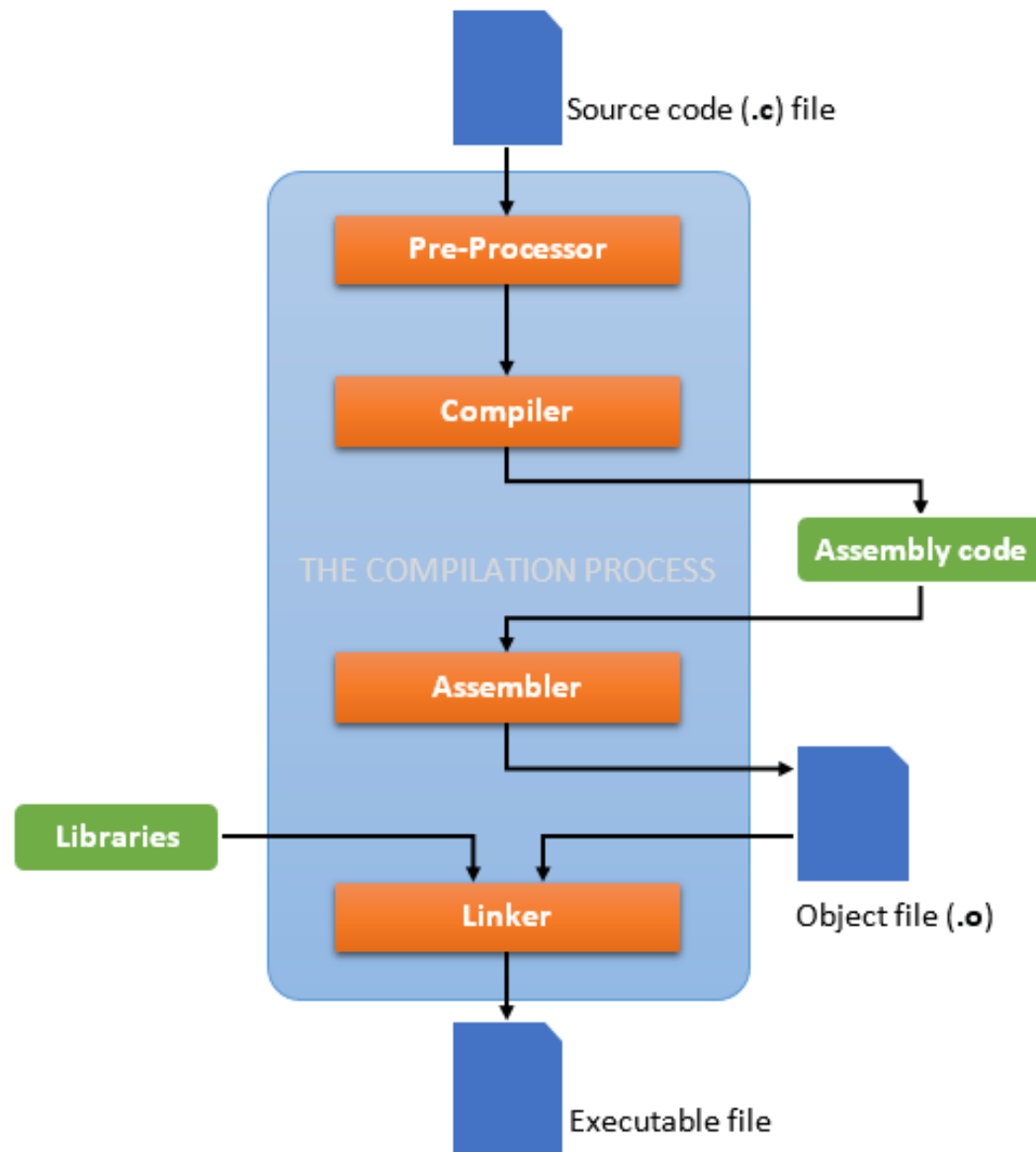
STEP 4 - LINKER

- The linker creates the final executable, in binary, and can play two roles:
 - linking all the source files together, that is all the other object codes in the project. For example, if I want to compile test.c with another file called trial.c and make them into one single program, this is the step where the object code of trial.c (that is trial.o) will be linked to the test.c object code (test.o).
 - linking function calls with their definitions. The linker knows where to look for the function definitions in the *static libraries* or *dynamic libraries*.

STEP 4 - LINKER

- By default, after this fourth and last step, that is when you type the whole “`gcc test.c`” command without any options, the compiler will create an executable program called **a.out**, that we can run by typing “**./a.out**” in the command line.
- We can also choose to create an executable program with the name we want, by adding the “-o” option to the gcc command, placed *after* the name of the file or files we are compiling, and pressing enter

```
gcc test.c -o test
```



PRACTICE

- Sample C programs with gcc
- Makefiles in C

Perimeter.0

gcc -c
perimeter.c



perimeter.c

Perimeter.o
gcc -c
perimeter.c
↑
perimeter.c

length.o
gcc -c
length.c
↑
length.c

Perimeter.o
gcc -c
perimeter.c
↑
perimeter.c

Length.o
gcc -c
length.c
↑
length.c

area.o
gcc -c
area.c
↑
area.c

